ALGORITHM SPECIFICATION & ANALYSIS REPORT

Tarjan's Algorithm: Strongly Connected Components

Single-pass DFS using low-link values and an active recursion stack for optimal component extraction

1. Algorithm Code (Part I: State Variables & Core DFS)

Below is Part 1 of the implementation, defining the state arrays and the recursive `tarjan` function that computes low-link values and pops components:

```
import java.util.*;

public class TarjanAlgo {

    static int time = 0;

    static int[] disc;
    static int[] low;
    static boolean[] onStack;

    static Stack<Integer> stack = new Stack<>();
    static List<List<Integer>> graph;

    static void tarjan(int u) {
        disc[u] = low[u] = time++;
        stack.push(u);
        onStack[u] = true;
        for (int v : graph.get(u)) {

            if (disc[v] == -1) {
                tarjan(v);
                low[u] = Math.min(low[u], low[v]);
            } else if (onStack[v]) {
                low[u] = Math.min(low[u], disc[v]);
            }
        }

        if (low[u] == disc[u]) {

            System.out.print("SCC: ");

            int v;

            do {
                v = stack.pop();
                onStack[v] = false;

                System.out.print((char) ('A' + v) + " ");

            } while (v != u);

            System.out.println();
        }
    }
```

Root Condition: A node u is the root of an SCC if and only if `low[u] == disc[u]`. When triggered, popping the stack down to u isolates that exact SCC.

1. Algorithm Code (Part II: Graph Setup & Invocation)

Below is Part 2 of the implementation, constructing the 7-node directed graph and invoking the search over all disconnected components:

```
    public static void main(String[] args) {

        int n = 7;

        graph = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        // A -> B
        graph.get(0).add(1);

        // B -> C, D
        graph.get(1).add(2);
        graph.get(1).add(3);

        // C -> A
        graph.get(2).add(0);

        // D -> E
        graph.get(3).add(4);

        // E -> F
        graph.get(4).add(5);

        // F -> D, G
        graph.get(5).add(3);
        graph.get(5).add(6);

        // G -> G
        graph.get(6).add(6);

        disc = new int[n];
        low = new int[n];
        onStack = new boolean[n];

        Arrays.fill(disc, -1);

        for (int i = 0; i < n; i++) {
            if (disc[i] == -1) {
                tarjan(i);
            }
        }
    }
}
```

> **Single Traversal Advantage:** Tarjan avoids creating a transposed graph, saving memory and executing in a single DFS pass.

2. Output & Output Explanation

```
SCC: G SCC: F E D SCC: C B A
```

Tarjan's algorithm prints components in reverse topological order (sinks first, sources last):

- **SCC: G:** Node G has a self-loop (`G → G`) and no outgoing edges to unvisited vertices. Because `low[G] == disc[G] == 6`, it pops immediately as a terminal component.
- **SCC: F E D:** Traversal reaches F, which discovers back-edge `F → D`. Since D is active on the stack, `low[F]` is updated to `disc[D] == 3`. At D, `low[D] == disc[D] == 3` triggers the pop of F, E, and D.
- **SCC: C B A:** Traversal reaches C, which detects back-edge `C → A` (where `disc[A] == 0`). This propagates `low = 0` back to B and A. At A, `low[A] == disc[A] == 0` pops C, B, and A.

3. Detailed Code Walkthrough

Variable / Routine	Mechanics & Implementation Purpose
`disc[u]`	Discovery timestamp of vertex u. Initialized to `-1`; assigned strictly increasing values as nodes are visited.
`low[u]`	Lowest discovery time reachable from u via its subtree and at most one back-edge to an active stack node.
`onStack[u]`	Maintains membership in the active DFS branch to distinguish valid back-edges from cross-edges to closed SCCs.
Tree-edge branch	`if (disc[v] == -1)`: Recurses into unvisited node v; updates `low[u] = Math.min(low[u], low[v])`.
Back-edge branch	`else if (onStack[v])`: Updates `low[u] = Math.min(low[u], disc[v])`. Cross-edges are safely ignored.
Component extraction	When `low[u] == disc[u]`, pops elements from `stack` and sets `onStack = false` until u is popped.

4. Algorithmic Foundation & Working Principle

Tarjan's algorithm builds upon DFS spanning forest classifications and subtree reachability:

- **Low-Link Reachability:** A node's low-link value represents the earliest-discovered ancestor reachable through its subtree. Vertices with the same low-link value belong to the same directed cycle.
- **Stack Invariant:** The stack contains vertices in discovery order. Nodes remain on the stack until their entire SCC is resolved, ensuring no components are prematurely split.

TIME COMPLEXITY
Θ(V + E)
Single pass over each node and edge.

AUXILIARY SPACE
Θ(V)
disc, low, onStack arrays and stack.

TRAVERSAL MODEL
Single Pass
No graph transposition or edge copying required.

5. Practical Uses & Comparative Advantages

Key applications in production software systems:

- **Low-Memory Settings:** Requiring only $O(V)$ auxiliary space rather than $O(V + E)$ makes Tarjan optimal for massive graphs that fit tightly in RAM.
- **Articulation Points & Bridges:** The discovery and low-link framework generalizes directly to finding biconnected components in undirected networks.
- **Deadlock & Cycle Resolution:** Quickly pinpoints cyclical deadlocks in dependency managers and thread wait graphs.

6. Real-World Industry Use Cases

Industry Domain	Operational Context	Tarjan's Application
Package Managers (e.g., npm, Cargo)	Dependency resolution during library installs.	Detects circular dependency loops on-the-fly and warns developers before publishing packages.
Build Systems (e.g., Bazel, Gradle)	Validating task execution DAGs.	Ensures build rules do not form cyclic dependencies, isolating cycle members for compilation diagnostics.
Database Query Engines	Resolving recursive Common Table Expressions (CTEs).	Verifies recursion termination bounds before executing costly distributed joins.
Autonomous Mesh Networks	Router biconnectivity and clustering.	Clusters routers into autonomous zones where internal communication remains guaranteed without border gateways.